

OS Lab Record - fork(), exec(), wait() Programs

Program 1: One Process (Before fork())

```
saketh@BHARGAVI-PC:~/OSLAB/fork$ for prog in program1 program2 program3 program4 program5 program6 program7 program8 output; do ./$prog; done
PID=1536
Hello
Hello
I am Parent
I am Child
Parent PID = 1541
Child PID = 1542
Child PID = 1542
Parent PID = 1222
Child Executing
Parent Executing
PID = 1545
PID = 1547
PID = 1548
PID = 1546
Parent Waiting...
Child executing ls...
PID = 1551
PID = 1555
PID = 1553
PID = 1556
PID = 1554
PID = 1557
PID = 1552
PID = 1558
-bash: ./output: No such file or directory
total 168
-rwxr-xr-x 1 saketh saketh 16000 Jul 30 09:16 program1
-rw-r--r-- 1 saketh saketh 92 Jul 29 03:46 program1.c
-rw-r--r-- 1 saketh saketh 92 Jul 30 09:49 program1.c.save
-rwxr-xr-x 1 saketh saketh 16000 Jul 30 09:22 program2
-rw-r--r-- 1 saketh saketh 89 Jul 29 03:51 program2.c
-rw-r--r-- 1 saketh saketh 90 Jul 30 09:46 program2.c.save
-rwxr-xr-x 1 saketh saketh 16000 Jul 29 03:53 program3
-rw-r--r-- 1 saketh saketh 167 Jul 29 03:53 program3.c
-rwxr-xr-x 1 saketh saketh 16088 Jul 29 03:55 program4
-rw-r--r-- 1 saketh saketh 282 Jul 29 03:55 program4.c
-rwxr-xr-x 1 saketh saketh 16040 Jul 29 04:01 program5
-rw-r--r-- 1 saketh saketh 220 Jul 29 03:57 program5.c
-rwxr-xr-x 1 saketh saketh 16040 Jul 29 04:03 program6
-rw-r--r-- 1 saketh saketh 120 Jul 29 04:02 program6.c
-rwxr-xr-x 1 saketh saketh 16040 Jul 29 04:05 program7
-rw-r--r-- 1 saketh saketh 226 Jul 29 04:04 program7.c
```

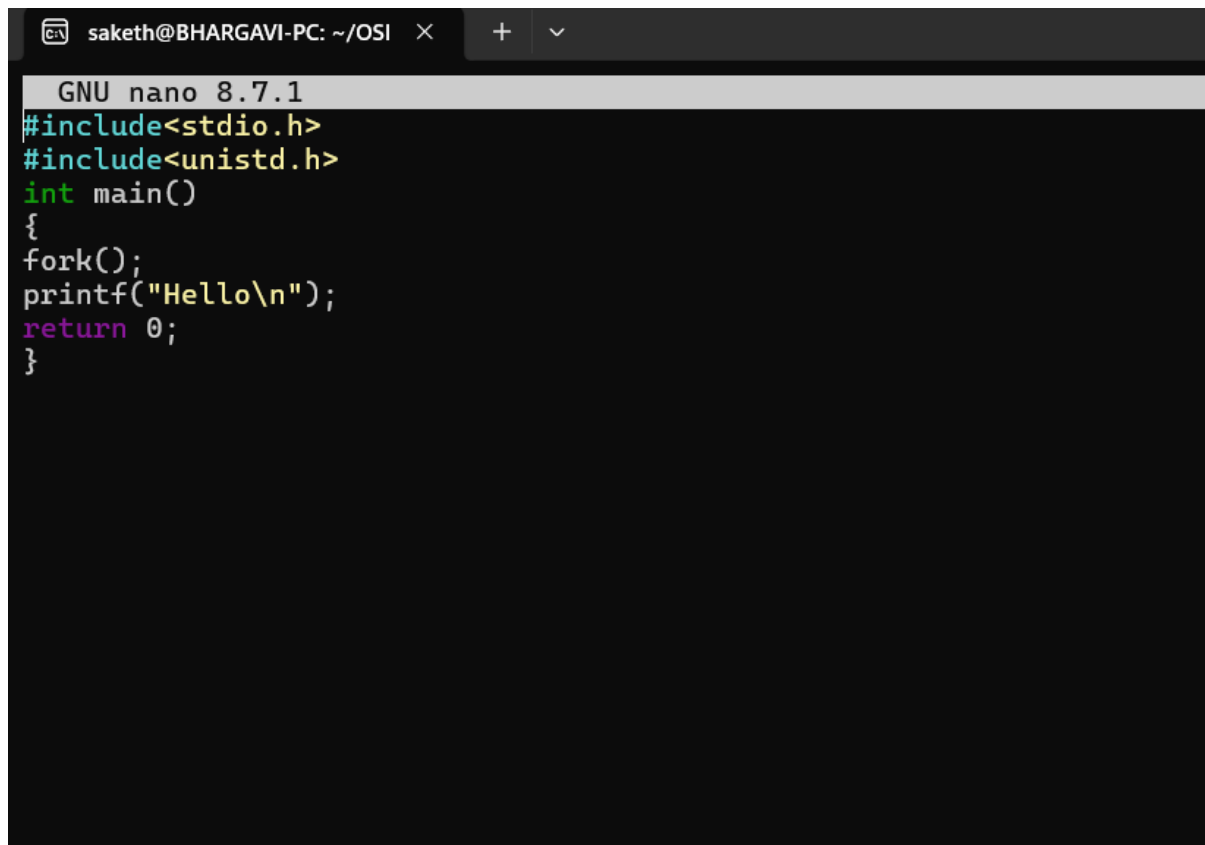
Program 2: fork() Creates a Child

```

1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 { pid_t pid; pid = fork();
5 }
6 mkdir OSLAB
7 cd OSLAB
8 mkdir fork
9 mkdir OSLAB
10 cd OSLAB
11 mkdir fork
12 cd fork
13 gedit program1.c
14 gedit ex1.c
15 sudo apt update
16 sudo apt install gedit -y
17 gedit program1.c
18 mkdir OSLAB
19 cd OSLAB
20 cd fork
21 gcc program5.c -o program5
22 ./program5
23 gedit program6.c
24 gcc program6.c -o program6
25 ./program6
26 gedit program7.c
27 gcc program7.c -o program7
28 ./program7
29 o
30 gedit program8.c
31 gcc program8.c -o program8
32 ./program8
33 mkdir OSLAB
34 cd fork
35 cd OSLAB
36 mkdir fork
37 cd fork
38 ./program1
39 ./program1.c
40 gedit program1.c
41 ./program1.c
42 ./program1
43 gcc program1.c -o program1
44 sudo apt install gcc
45 sudo apt update
46 sudo apt install build-essential
47 gcc --version
48 gcc program1.c -o program1
49 ./program1
50 gedit program2.c
51 ./program2
52 gcc program2.c -o program2
53 ./program2
54 gcc program2.c -o program2
55 ./program2
56 gedit program3.c
57 gcc program3.c -o program3
58 ./program3
59 gedit program4.c
60 gcc program4.c -o program4
61 ./program4
62 gedit program5.c
63 cd OSLAB
64 cd fork
65 gcc program1.c -o program1
66 ./program1
67 gcc program2.c -o program2
68 gcc program2.c -o program2
69 ./program2
70 ./program3
71 ./program4
72 ./program5
73 ./program6
74 ./program7
75 o
76 ./program8
77 nano program1.c
78 cd OSLAB
79 cd fork
80 ./program1
81 ./program2
82 ./program3
83 ./program4
84 ./program5
85 ./program6
86 ./program7
87 ./program8
88 nano program2.c
89 nano program3.c
90 ./program3
91 cd OSLAB
92 cd OSLAB
93 cd fork
94 nano program3.c
95 nano program4.c
96 nano program5.c
97 nano program6.c
98 nano program7.c
99 nano program78.c
100 nano program8.c
101 history

```

Program 3: Parent vs Child

A screenshot of a terminal window. The title bar shows 'saketh@BHARGAVI-PC: ~/OSI' with a close button. The terminal content shows the GNU nano 8.7.1 editor with a C program. The code includes <stdio.h> and <unistd.h>, defines a main function that calls fork(), prints 'Hello\n', and returns 0. The code is color-coded: #include is blue, int is green, return is purple, and strings are yellow.

```
GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>
int main()
{
fork();
printf("Hello\n");
return 0;
}
```

Program 4: Display Both PIDs

```
saketh@BHARGAVI-PC: ~/OSI × + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
        printf("I am Child\n");
    else
        printf("I am Parent\n");
    return 0;
}
```

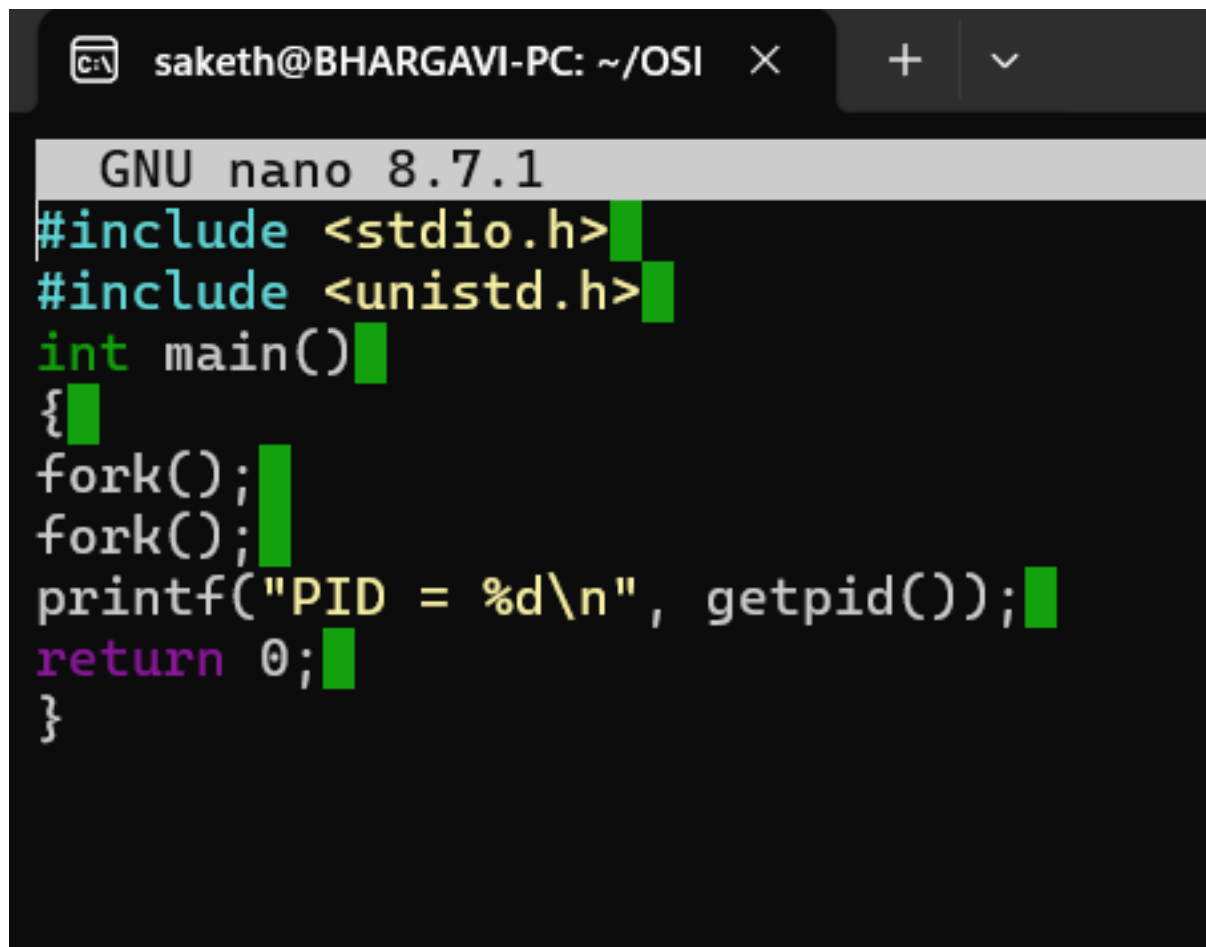
Program 5: Parent Waits for Child

```
saketh@BHARGAVI-PC: ~/OSI × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

Program 6: Multiple fork() Calls

```
saketh@BHARGAVI-PC: ~/OSI × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Executing\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Executing\n");
    }
    return 0;
}
```

Program 7: Child Executes Another Program (exec())



The image shows a terminal window with a dark background. At the top, a tab is labeled 'saketh@BHARGAVI-PC: ~/OSI'. Below the tab bar, the title of the editor is 'GNU nano 8.7.1'. The code being edited is a C program that includes `<stdio.h>` and `<unistd.h>`. The `main` function calls `fork()` twice in sequence, then prints the process ID using `getpid()`, and finally returns 0. The code is color-coded: `#include` is yellow, `int` is green, `return` is purple, and other keywords and symbols are white. Green cursor blocks are visible at the end of each line.

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
fork();
fork();
printf("PID = %d\n", getpid());
return 0;
}
```

Program 8: fork() Inside a Loop

```
saketh@BHARGAVI-PC: ~/OSI × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child executing ls...\n");
        execl("/bin/ls", "ls", "-l", NULL);
    }
    else
    {
        printf("Parent Waiting...\n");
    }
    return 0;
}
```

Compilation/History


```
saketh@BHARGAVI-PC: ~/OSI × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    int i;
    for(i=0;i<3;i++)
        fork();
    printf("PID = %d\n",getpid());
    return 0;
}
```

Program Outputs

```
saketh@BHARGAVI-PC: ~/OSI x + v
GNU nano 8.7.1 program1.c *
#include<stdio.h>
#include<unistd.h>
int main()
{
printf("PID=%d\n",getpid());
return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^G Location   M-U Undo     M-A Set Mark  M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo     M-C Copy      ^B Where Was
```